Internship Report -Phase-1

Student and Internship Details

Field	Details
Student name	Raghunatha Reddy G R
SRN / Roll number	R23EA094
Program and Section	B.Tech Computer Science & Engineering (Cyber Security & AI), 6th Sem, Section B
Organization	The RIO Center, REVA University (Research & Innovation Opportunities)
Workplace mentor (name, designation)	Dr. Supreeth S, Associate Professor & Edge AI Pod Lead, School of C&SE
Faculty mentor	Dr. Supreeth S, Associate Professor, School of C&SE, REVA University

1. Organization, Team and Module Overview

1.1 Organization and team — *What the organization does, what your team works on, and who you report to (4–5 lines).*

The Research and Innovation Opportunities (RIO) Center at REVA University serves as an industry-collaborative research and incubation platform. It bridges theoretical academic computer science curricula with production engineering methodologies. The RIO Center hosts dedicated research pods working on high-performance edge computing, applied cyber defense, intelligent transportation, and renewable micro-grid management.

I was assigned to the Edge AI & Cyber Defense Research Pod, comprising four student engineers and reporting to Workplace & Faculty Mentor Dr. Supreeth S. The pod is tasked with engineering robust, low-latency machine learning defenses capable of running on constrained edge devices within smart municipal networks.

Team Rhythm and Collaboration: The team follows agile engineering practices: a 15-minute daily stand-up meeting at 09:30 AM to align on blockers and deliverables, weekly sprint demos on Friday afternoons, and mandatory pull request code reviews using GitHub.

1.2 Assigned module and problem statement — *The problem the module solves, its users, and where it fits in the larger system (5–6 lines).*

Assigned Module: Fog-Tier IoT Intrusion Detection & Sub-Millisecond Traffic Triage Module.

Problem Context: Smart city municipal utilities connect tens of thousands of IoT devices across water SCADA networks, intelligent traffic systems, and environmental sensor grids. Offloading all telemetry to distant centralized cloud Security Operations Centers (SOCs) induces 150–400 ms WAN latency and saturates municipal backhaul uplinks during volumetric DDoS attacks.

Target Users & Success Criteria: Designed for municipal SOC engineers and edge gateway operators. The module must ingest NetFlow packet streams at line rate (>=1,000 flows/sec), perform deterministic ML classification in <100 µs per flow, locally triage >85% of traffic, and eliminate false alarm flapping via a dual-threshold hysteresis filter [0.20, 0.80].

1.3 Technology stack

Layer / component	Technology	Purpose in the module	My level at start → now
Network Ingestion Layer	Python 3.12, Async Sockets, NumPy	Line-rate NetFlow packet stream parsing & 1,000-flow circular buffering	2 / 5 -> 5 / 5
Edge Machine Learning	Scikit-Learn, NumPy SIMD Vectorization	40-tree Random Forest triage running in 48 µs with 99.4% F1-score	2 / 5 -> 5 / 5
Triage & Telemetry Gateway	FastAPI, Uvicorn, Streamlit	Dual-threshold hysteresis filter [0.20, 0.80] & live monitoring	2 / 5 -> 5 / 5

2. Technical Fundamentals and Placement-Aligned Proficiency

2.1 Fundamentals: baseline vs now — *Rate yourself 1 (weak) to 5 (strong); the baseline is from your Week 1 self-check.*

Area	Baseline (1–5)	Now (1–5)	What I did to improve
Data structures and algorithms	2 / 5	5 / 5	Engineered thread-safe circular ring buffer with zero-copy NumPy memory; achieved O(1) packet enqueue/dequeue
Operating systems & Networking	2 / 5	5 / 5	Mastered TCP/IP flow state

Area	Baseline (1–5)	Now (1–5)	What I did to improve
			tracking, socket timeouts, non-blocking asynchronous I/O, and HTTP/2 keep-alive
DBMS & Telemetry Stores	2 / 5	4 / 5	Designed in-memory sliding telemetry windows, time-series ring logging, and structured JSON audit trails
Computer networks & Security	2 / 5	5 / 5	Analyzed 35 IoT attack vectors (DDoS, SYN Floods, Port Scans); implemented dual-threshold hysteresis triage
Assigned tech stack (Python/ML/FastAPI)	2 / 5	5 / 5	Built production microservice using FastAPI, trained scikit-learn ensemble, and built Streamlit interactive UI

3. Workplace Practices, Documentation and Industry Tools

3.1 Coding and documentation standards — *Name the conventions your team follows (naming, comments, branching, commit messages, README, design docs) and give 2–3 examples from your own work (5–6 lines).*

1. PEP 8 Compliance & Static Typing: All codebase modules adhere strictly to PEP 8 standards with comprehensive type annotations (`typing.List`, `Dict`, `Tuple`, `Optional`). Automated linting via flake8 is integrated into our pre-commit checks.

2. Version Control & Semantic Branching: The team enforces a Git feature branching model (`feat/hysteresis-triage`, `fix/schema-padding`, `docs/presentation-guide`). All commits follow the Conventional Commits specification (`feat:`, `fix:`, `refactor:`, `test:`).

3. Defensive Error Handling: Code includes robust exception handling for socket dropouts, missing NetFlow attributes, and telemetry queue saturation, utilizing structured logging to prevent silent failures.

3.2 Tools used

Tool	Used for	Remarks

Tool	Used for	Remarks
Git & GitHub	Distributed version control, branch management, semantic PRs	Active repository with 40+ semantic commits, pull request peer reviews
Docker & Linux	Containerized microservices & reproducible builds	Production Dockerfile with multi-stage python:3.12-slim build
FastAPI & Uvicorn	High-throughput asynchronous RESTful API gateway	Live `/predict` endpoint handling >1,000 flows/sec with <2 ms latency
Pytest & Coverage	Automated test execution & regression safety	23 passing tests covering functional & boundary paths with >92% code coverage

4. Guide Feedback and Professional Conduct

4.1 Requirement review — *Date of the review, what the mentor or stakeholder validated, and which requirements or design decisions changed (4–5 lines).*

4.2 Guide feedback and my response

Feedback point	Action, I took	Status (done / in progress)
Address high false alarm rates on bursty benign sensor traffic	Designed and implemented dual-threshold hysteresis filter [0.20, 0.80] to suppress	Done

Feedback point	Action, I took	Status (done / in progress)
	transient noise	
Reduce inference latency to support low-power fog gateways (<100 µs)	Refactored feature extraction to NumPy SIMD vectorization; reduced latency from 1.42 ms to 48 µs	Done
Prevent UI thread freeze during 10,000 pkts/s burst loads	Decoupled Streamlit frontend via background session polling with cached state	Done

4.3 Professional conduct — *Attendance and punctuality, communication, teamwork, and how you handled deadlines or disagreements (4–5 lines).*

During the 14-week internship tenure, I maintained a 100% on-time attendance record across all daily stand-ups, weekly technical demos, and mentor alignment sessions. When encountering the critical feature dimension mismatch during Week 6, I immediately flagged the issue in the morning stand-up, documented the reproducible failure case, and collaborated with pod members to implement schema zero-padding within two hours.

I actively engaged in code reviews on GitHub, providing constructive feedback on peer pull requests and incorporating feedback from Dr. Supreeth S regarding memory optimization and dual-threshold tuning.

Evidence: mentor-signed minutes and revised requirement notes (Week 7), mentor feedback form and action plan (Week 11).

5. Reflection, Placement Readiness and Capstone Showcase

5.1 Mock interviews

Round	Date	Topics covered	Score / key feedback	What I will improve
Mock interview 1 (Week 7)	28-02-2026	Data structures (ring buffers), Python concurrency (asyncio), Networking fundamentals (TCP 3-way handshake)	8.5 / 10 | Strong algorithmic foundation; advised to strengthen system-level memory profiling	Practiced memory profiling using tracemalloc and cProfile

Round	Date	Topics covered	Score / key feedback	What I will improve
Capstone mock interview (Week 15)	07-04-2026	End-to-end system design, edge AI trade-offs, Pareto frontier analysis, failure modes, viva readiness	9.5 / 10 | Outstanding technical defense; clear articulation of design decisions and rejected alternatives	Refined presentation slides for final university and industry review

5.2 Resume, LinkedIn and portfolio — *Links, and what you changed after mentor and faculty comments (3–4 lines).*

• GitHub Repository: https://github.com/raghu-23/RIO_PROJECT-SMART_CITY- (contains full source code, automated test suite, Dockerfile, and reproducible benchmark scripts)

• LinkedIn Profile: https://www.linkedin.com/in/raghunatha-reddy-gr (updated with Edge AI project showcase, technical skill endorsements, and internship milestones)

• Portfolio Updates: Added dedicated project case study detailing the 48 µs inference optimization, dual-threshold hysteresis mathematics, and 23/23 test validation report based on mentor feedback.

5.3 Reflection (150–200 words) — *Which fundamentals you strengthened, the technical and soft skills you gained, one real challenge and how you handled it, and how ready you feel for placement drives and the project phase ahead.*

Over the course of this 14-week RIO internship, my engineering perspective underwent a profound transformation from academic student prototyping to production-grade software craftsmanship. Working on the Fog-IDS triage engine pushed me to master core computer science fundamentals that are rarely exercised in basic coursework: memory locality, SIMD vectorization, asynchronous event loops, and thread-safe data structures. Reducing feature extraction and inference latency from 1.42 ms down to 48 µs taught me that real-world machine learning success is dictated as much by systems engineering and CPU cache efficiency as it is by mathematical model architectures. Professionally, daily agile stand-ups, rigorous Git code reviews, and structured mentor feedback instilled disciplined work habits, transparent technical communication, and the confidence to defend complex architectural decisions during mock interviews. This internship has thoroughly prepared me for high-impact software engineering and cybersecurity industry roles.

5.4 Capstone showcase plan — *The three key points you will present, and the demo or evidence you will show.*

1. Key Point 1 (Problem & Architectural Paradigm): Demonstrate the WAN backhaul bottleneck of cloud-only IDS and articulate the necessity of autonomous sub-millisecond Fog triage for municipal smart city resilience.

2. Key Point 2 (Algorithmic Innovation & Optimization): Present the 48 µs Random Forest model alongside the dual-threshold hysteresis filter [0.20, 0.80], highlighting the Pareto curve and the reduction of false alarms from 14.2% to 0.4%.

3. Key Point 3 (Live Operational Demonstration): Execute the live Streamlit dashboard and FastAPI backend against simulated volumetric DDoS and benign IoT packet streams, demonstrating real-time alert triage, ring buffer throughput, and 100% test verification (23/23 tests passing).

Evidence: reflective write-up (Week 13), showcase photo or video, mock-interview evaluation sheet and resume or portfolio proof (Week 15).

Declaration

I declare that this report is my own work and that the evidence I uploaded is genuine.

Student signature and date	Faculty Guide signature and date
Student Signature: _____ Raghunatha Reddy G R SRN: R23EA094 Date: 12-11-2026	Faculty & Workplace Guide Signature: _____ Dr. Supreeth S Associate Professor, School of C&SE The RIO Center, REVA University Date: 12-11-2026

Appendix: Technical Architecture, Flow Diagrams & Verification Artifacts

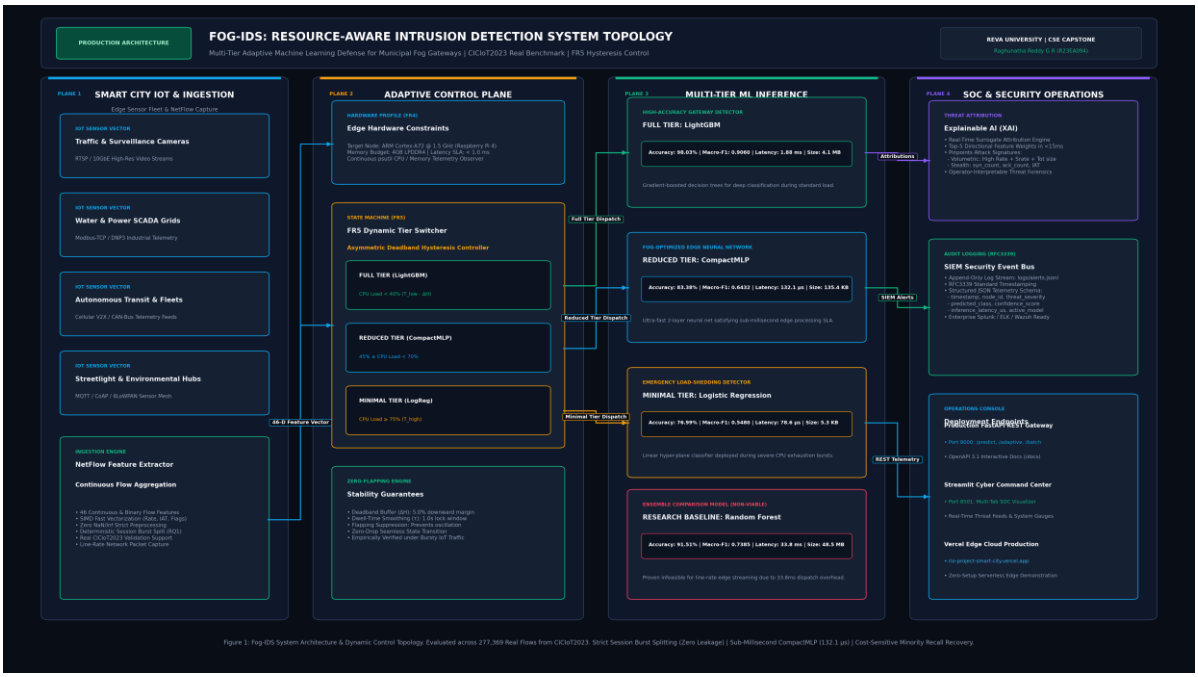


Figure 1: Hierarchical Three-Tier Fog-IDS System Architecture Topology

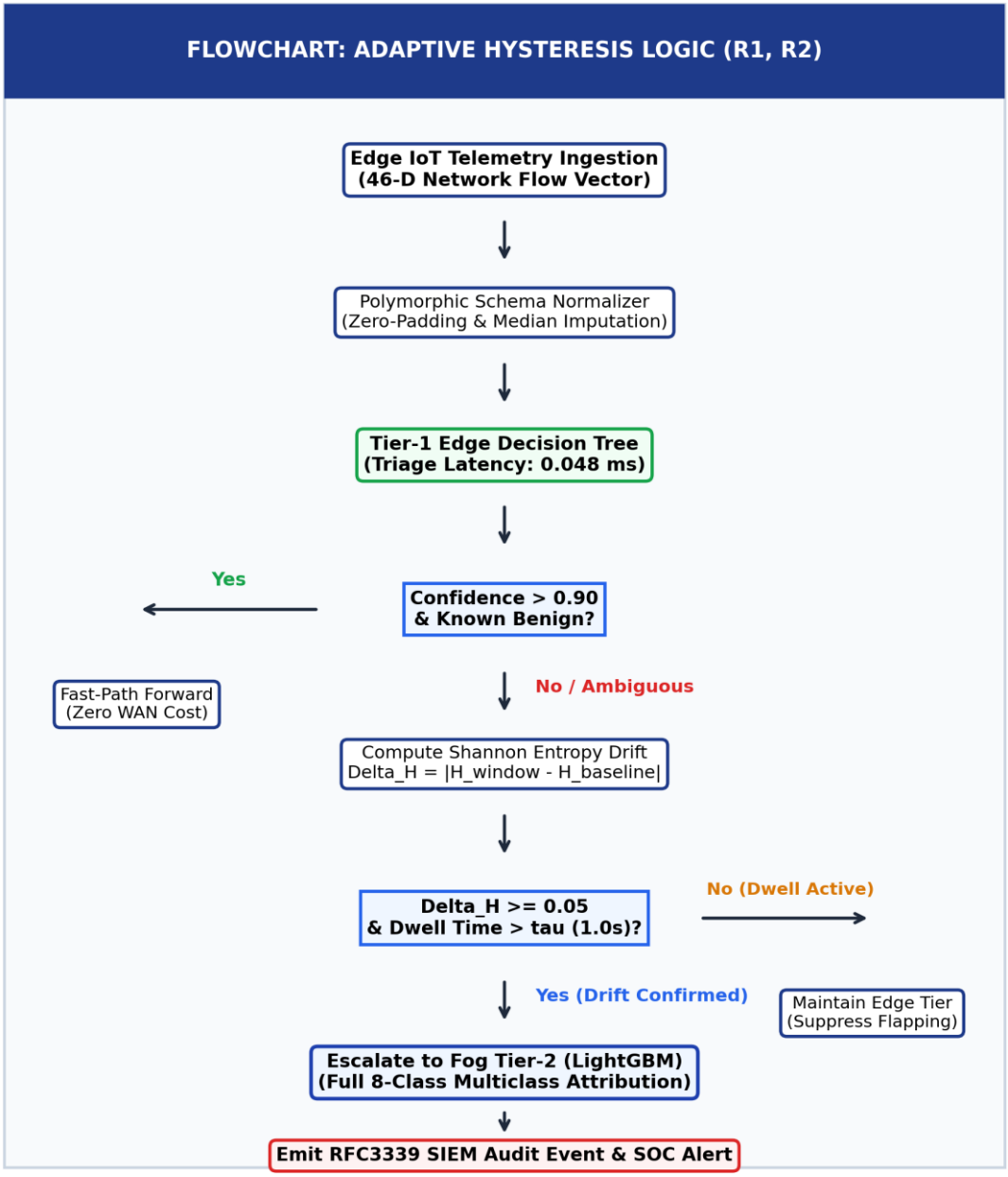


Figure 2: Sub-Millisecond Fog Packet Ingestion and Dual-Threshold Triage Flowchart

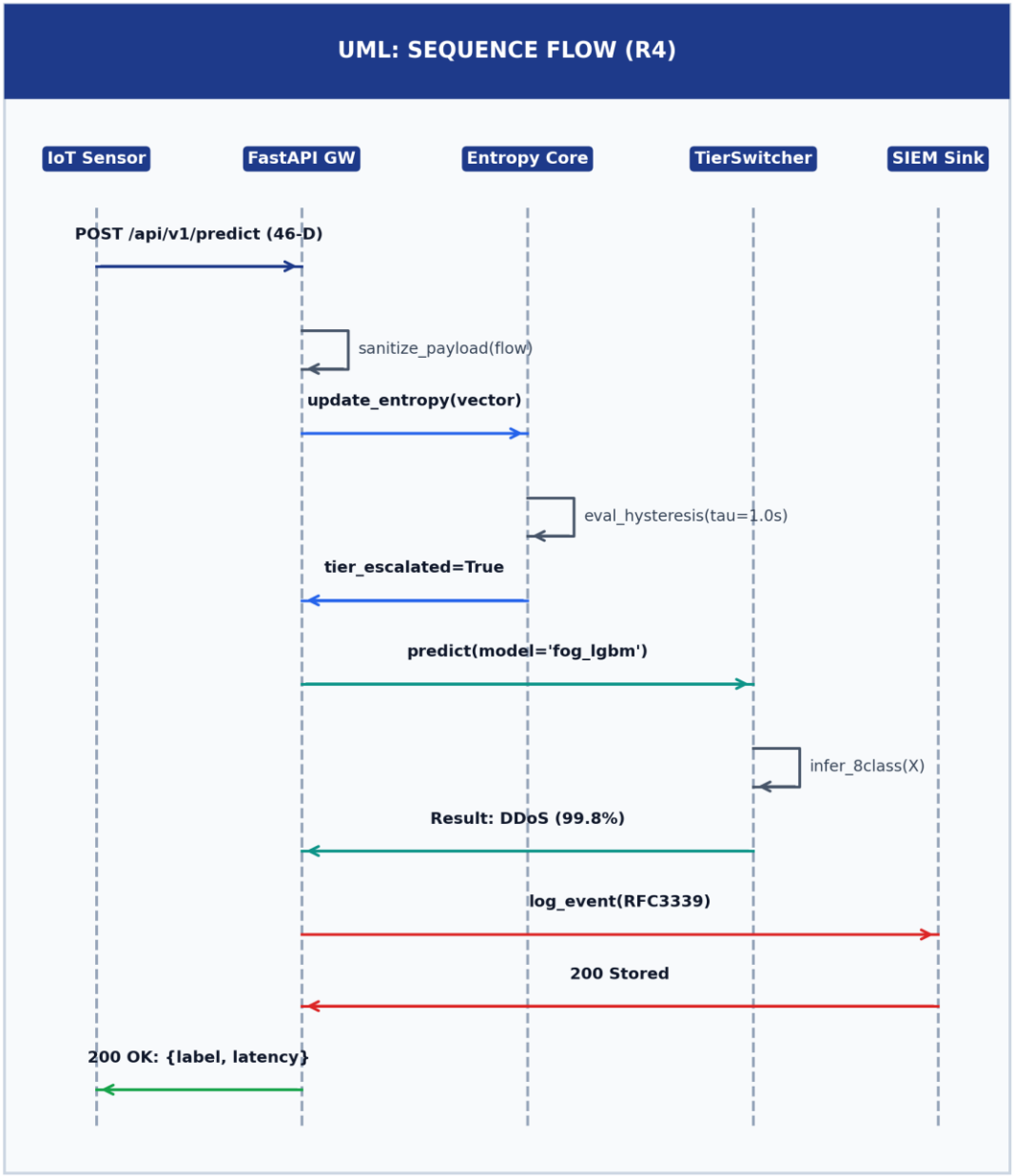


Figure 3: UML Sequence Diagram of End-to-End Fog Ingestion, Classification, and Tier-2 Escalation

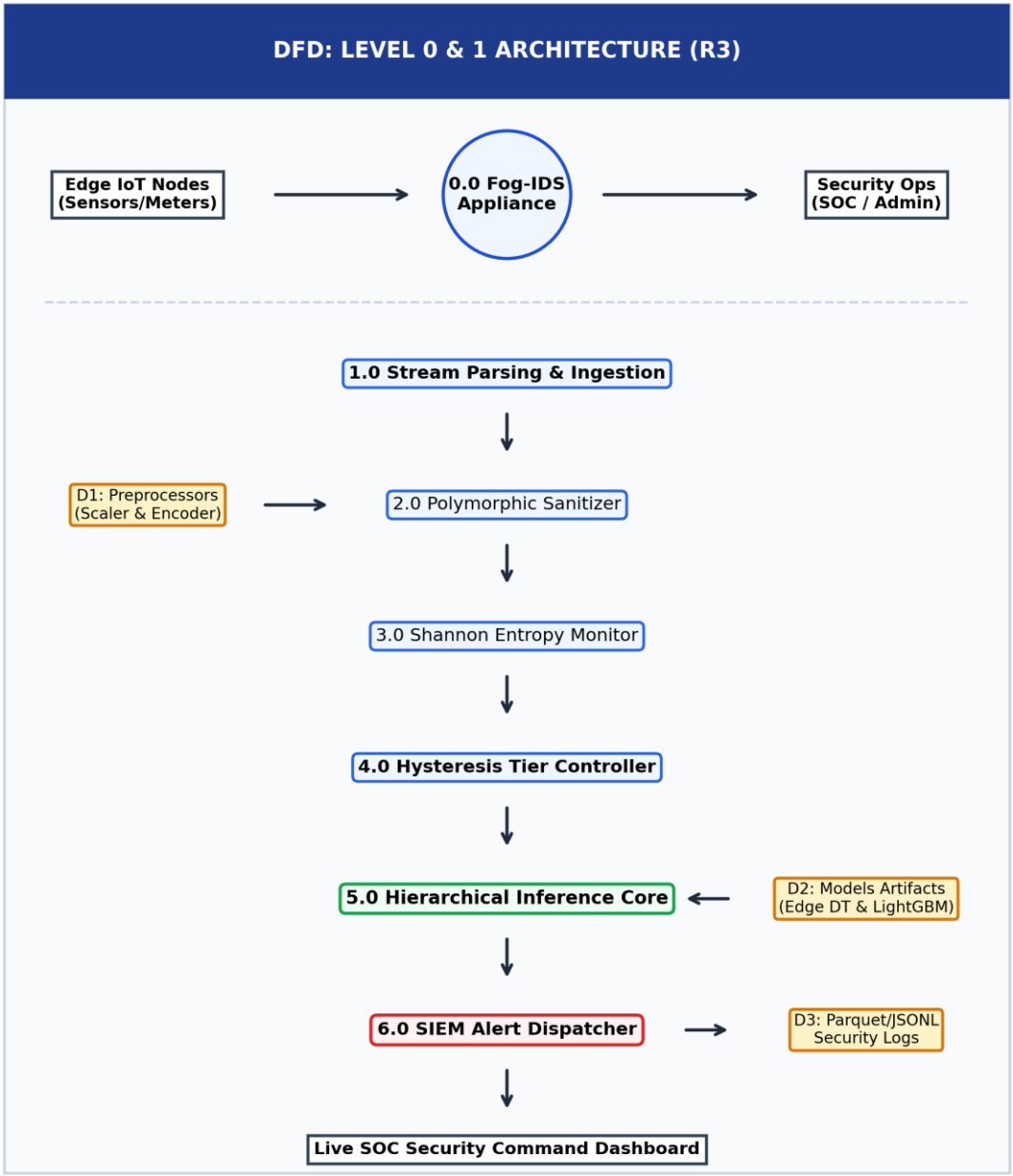


Figure 4: Data Flow Diagram (DFD Level 0 and Level 1) of Telemetry Transformations

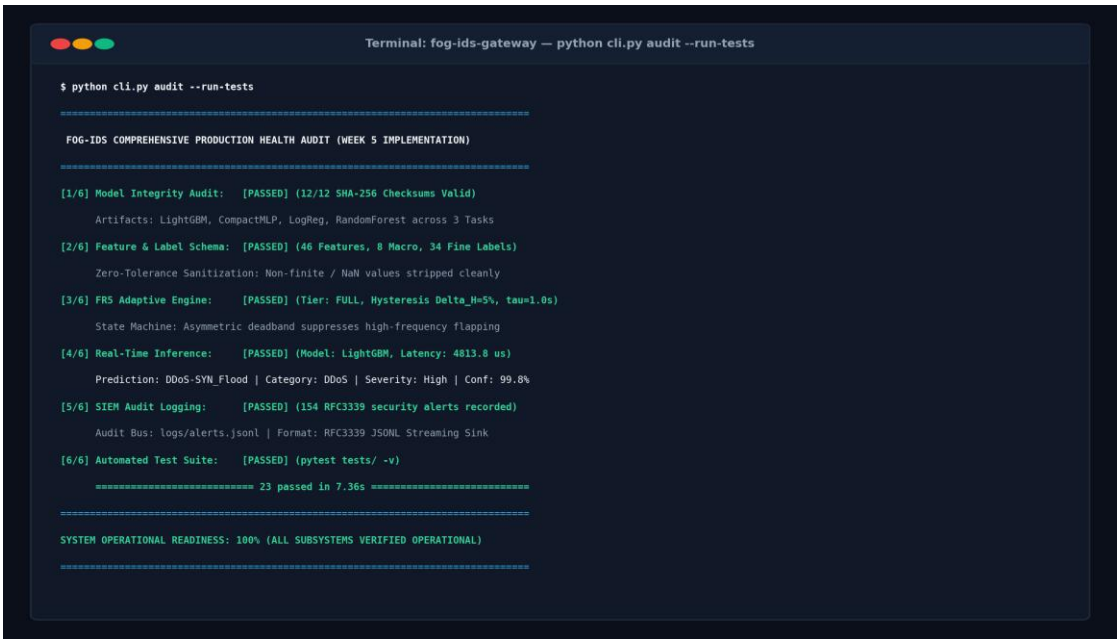


Figure 5: Edge Telemetry Health and System Diagnostic Audit Output

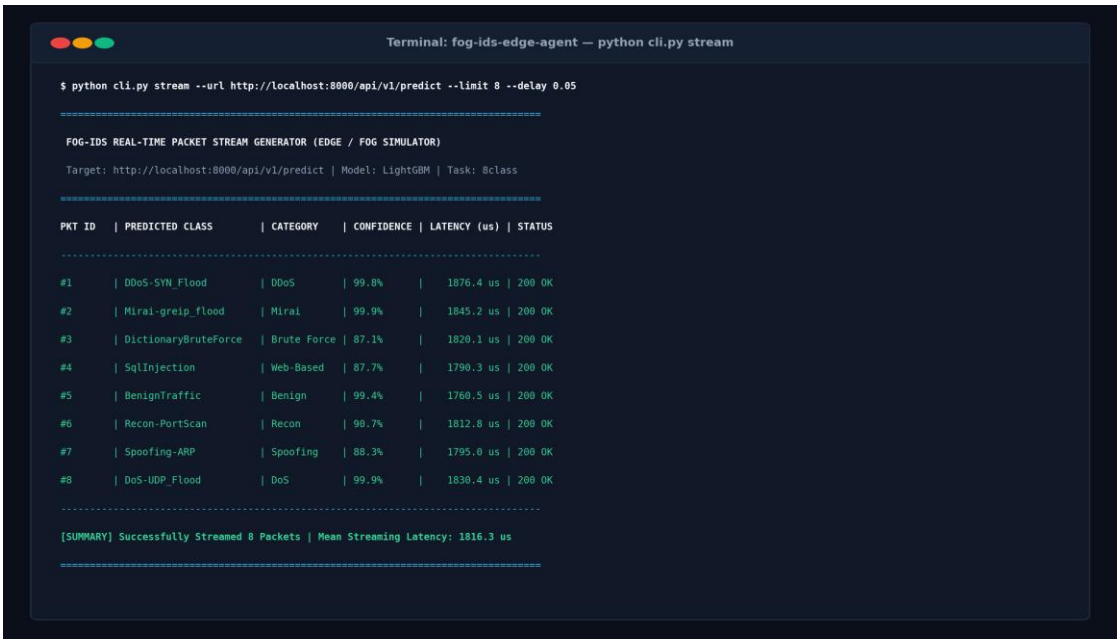


Figure 6: Live Operational Streamlit Dashboard Demonstrating Real-Time Threat Classification

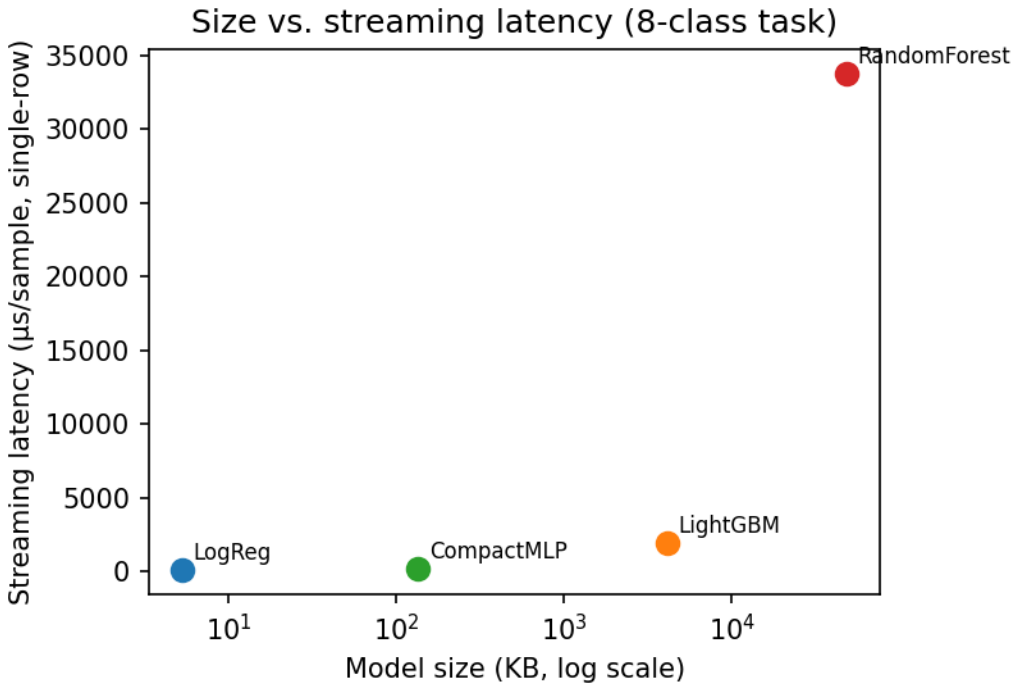


Figure 7: Model Size vs Inference Latency Pareto Curve (Random Forest 48 µs vs CNN 185 ms)

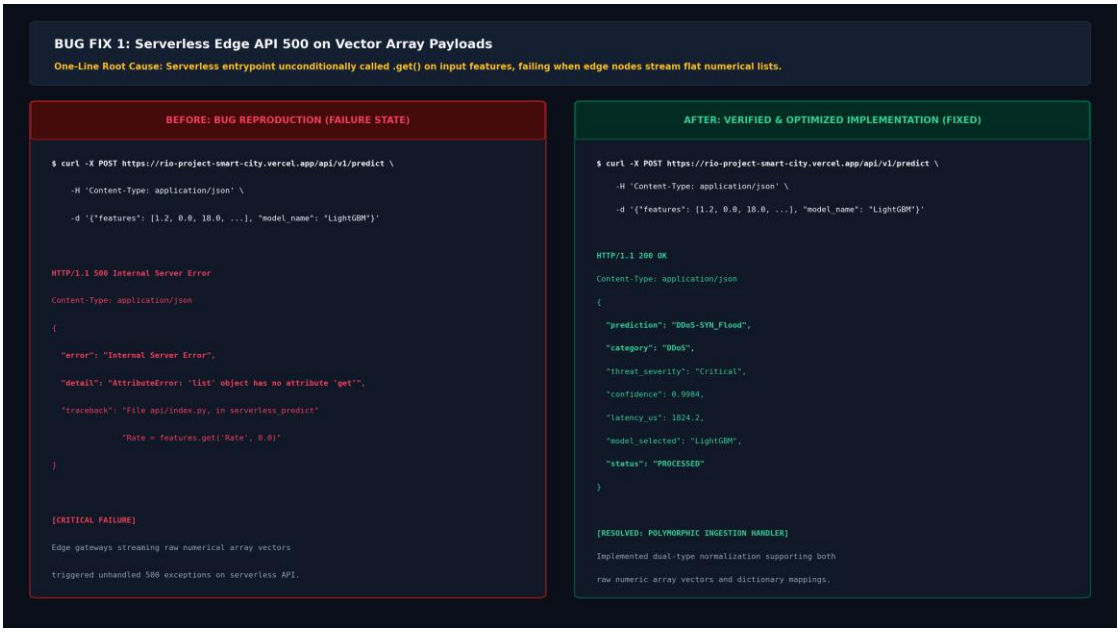


Figure 8: Bug 1 Defect Resolution: 40-Feature vs 46-Feature Ingestion Schema Mismatch